

Adaptive Auto-Harness: Sustained Self-Improvement for Agentic System Deployment on Open-Ended Task Streams

Anonymous ACL submission

Abstract

Auto-harness systems such as A-Evolve, GEPA, and Meta-Harness improve LLM agents by optimizing prompts, skills, tools, memories, and supporting infrastructure from execution feedback, but they are typically evaluated on fixed offline benchmarks. Real deployments instead present open-ended task streams: histories grow without a fixed endpoint, heterogeneous tasks require different harnesses, and problem distributions shift over time. These challenges make a single repeatedly and densely updated harness brittle, causing performance degradation as accuracy peaks early and then declines. This motivates sustained harness construction with task-wise adaptation. We introduce Adaptive Auto-Harness, a framework and system for such streams. The framework decomposes the gap to an oracle harness into evolution loss and adaptation loss. The system addresses these losses with a stateful multi-agent evolver, a harness tree with solve-time routing, and human-steering hooks for cases where history lacks the needed signal. Across prediction-market, security-competition, and event-forecasting streams, Adaptive Auto-Harness outperforms five existing auto-harness baselines and ablations attribute gains to better construction, routing, or targeted human steering.

1 Introduction

An agent’s harness, comprising the prompts, skills, tools, and supporting infrastructure that surround a fixed LLM, is a primary determinant of task-solving performance. *Auto-harness systems* such as A-Evolve (Lin et al., 2026), GEPA (Agrawal et al., 2025), and Meta-Harness (Lee et al., 2026) performs evolution to construct harness automatically from execution feedback, and report substantial gains on static offline benchmarks such as SWE-bench (Jimenez et al., 2024). However, online real-world deployment, whose tasks and evaluations follow a strict chronologic order, demands *sustained*

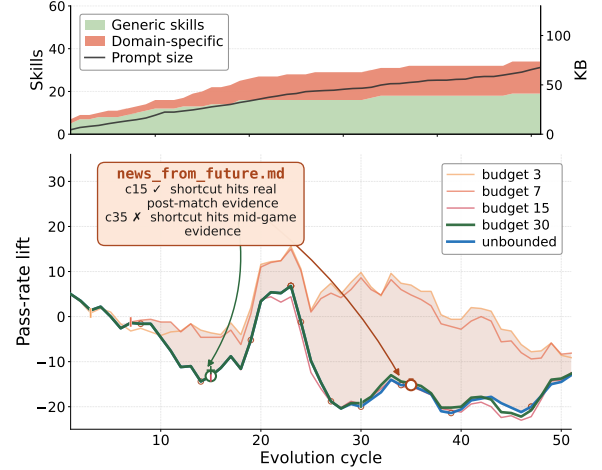


Figure 1: Harness overfits past-stream patterns. **Top:** unbounded-run skills grow from 12 generic to 34, prompt size grows from 2KB to 68KB. **Bottom:** pass-rate lift for evolution with fixed cycles on Polymarket trading tasks overtime.

self-improvement: a sustained improvement of the harness as the agent operates on an open-ended task stream rather than on a fixed evaluation set. Representative settings include prediction-market platforms with thousands of questions across diverse topics over weeks (Cheng et al., 2026), security competitions spanning over a decade of evolving challenges (Zhuo et al., 2025), and forecasting services handling cross-lingual queries from heterogeneous sources (Zeng et al., 2025).

A concrete deployment failure illustrates the gap. As shown in Figure 1, we perform A-evolve on polymarket trading tasks across weeks with different evolution budgets (evolution stops after N cycles). While evolution initially improves the harness (lift compared to no evolution), the runs with more budget accumulates source preferences and tools growth that consume context and bias later tasks. Across all five evolution-budget settings, every curve peaks early and declines, with fewer budget runs beating those with more evolutions,

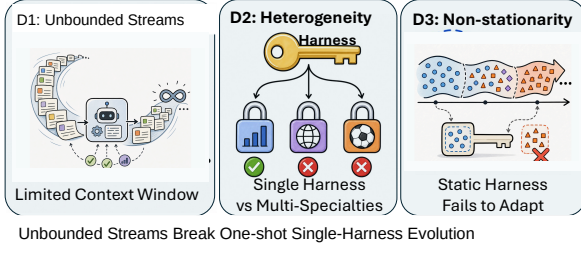


Figure 2: Three deployment dimensions in open-ended task streams.

which indicates achieving sustained improvement on open-ended task stream requires more than continual update with more cycles.

Therefore, as illustrated in Figure 2, we point out three deployment dimensions that distinguish the two settings. **(D1) Unbounded Streams.** The task stream has no fixed train/test cutoff or designated endpoint; feedback, trajectories, and harness state keep accumulating during deployment, creating heavy burden for the evolution. Existing auto-harness systems built around a single-agent evolver are bounded by a finite context window, leaving little room to approach the large and growing volume of trajectories that effective and generalizable harness would require to be built. **(D2) Task heterogeneity.** The stream contains tasks of varied types that demand different harness components. A prediction-market platform, for example, mixes politics, sports, and finance questions in the same hour, each calling for distinct sources, tools, and prompting patterns. This rationale parallels a more general phenomenon: a single fixed policy is rarely optimal across heterogeneous problems (Miao et al., 2025). Existing auto-harness systems analogously deploy a static dense harness across the stream, with no solve-time adaptation to the task at hand. **(D3) Distributional non-stationarity.** As the stream progresses, incoming tasks shift away from the experience the harness was last fitted on. A harness optimised for recent cycles therefore drifts out of fit for new tasks, even with rich historical experience. Closing this gap requires per-task contextual adaptation of the harness, not only continued historical fitting. Additional diagnostics are shown in Appendix A.

We address these three dimensions under a unified analytical framework and identify two root gaps that prior auto-harness systems overlook. This analysis motivates: (1) *Sustained auto-harness* (§3.3), which replaces stateless one-shot evolution with a stateful multi-agent system with cross-cycle

knowledge for better harness construction; and (2) *Solve-time adaptation* (§3.4), which adapts a task-relevant harness for each problem prior to solving, restoring per-task fit on a heterogeneous, drifting stream. Beyond the two axes, we further introduce a third axis: human-in-the-loop (HITL) for auxiliary steering of the harness to incorporate human insights and provisions that are absent from historical experience. Our contributions are:

- (1) **Deployment-regime analysis of open-ended agentic streams.** We formalise why static auto-harness evaluation is insufficient once tasks arrive as unbounded, heterogeneous, and non-stationary streams. The framework decomposes the gap towards the optimal harness into evolution loss and adaptation loss, providing guidance for auto-harness system designs.
- (2) **Adaptive Auto-harness system.** We introduce a stateful multi-agent evolver for sustained harness construction, a harness-tree router for solve-time adaptation, and structurally triggered human-in-the-loop hooks for regimes where historical experience is insufficient.
- (3) **Comprehensive empirical validation and diagnosis.** We evaluate on three temporal streams spanning prediction markets, security challenges, and event forecasting against other auto-harness systems. Beyond aggregated performance, we provide in-depth analysis and evidence for the proposed gaps, component ablations, and human-in-the-loop slice analyses.

2 Related Work

Continual Learning in Task Streams. Continual learning studies systems that learn from a sequence of tasks while retaining earlier capabilities (Buzzega et al., 2020; Wang et al., 2022b,a). Domain and test-time adaptation address distribution shift (Ben-David et al., 2010; Ganin et al., 2016; Wang et al., 2021; Liang et al., 2020), and mixture-of-experts methods route heterogeneous inputs to specialised components (Jacobs et al., 1991; Shazeer et al., 2017). These directions cover pieces of our D1–D3 setting, but they usually adapt model weights, classifiers, or expert modules. We study the harness-level analogue: prompts, skills, tools, memory, and infrastructure must be constructed from accumulated trajectories and selected for each incoming task.

Self-improving and self-evolving agents. The closest LLM-agent precedents to our setting are

systems that update the harness directly from execution feedback. A-Evolve (Lin et al., 2026) introduces a linear-chain evolver: each cycle reads batch trajectories and mutates prompts, skills, memory, and tools. GEPA (Agrawal et al., 2025) adds reflective Pareto prompt evolution using textual feedback rather than scalar reward. Meta-Harness (Lee et al., 2026) uses a growing filesystem archive and Claude Code as the proposer. Continual Harness (Karten et al., 2026) enables online adaptation within a single continuous deployment run through alternating action/refinement cycles. SkillOS (Ouyang et al., 2026) learns a skill curation policy via reinforcement learning, training a curator to select and refine reusable skills from repeated interactions.

3 Method

3.1 Problem Formulation

We consider tasks arriving in an open-ended stream $x_1, x_2, \dots, x_T$ with $x_t \sim P_t$, where P_t is the task distribution at time t and each x_t has a fixed ground truth $y(x_t)$. The agent observes the history of prior experience $\mathcal{H}_t = \{(x_i, r_i, \tau_i)\}_{i=1}^{t-1}$, where r_i is optionally the realized reward on task x_i and τ_i is the agent’s solving trajectory (actions, intermediate observations, tool calls). A *harness* $C = \varphi(\mathcal{H}_t)$, with $|C| \leq K$, is a bounded representation (prompts, skills, memory, and tools) with capacity budget K , where φ is the *evolver*: the agentic system that automatically transforms historic experience into the harness. A solver agent then acts via the policy $a_t \sim \pi(a \mid x_t, C_t)$, where π is the LLM-induced sampling distribution over actions conditioned on the task and harness.

3.2 Analytical Framework

The three deployment dimensions of §1 surface concrete failures, but they do not by themselves identify what an evolver should fix. To pinpoint the root causes, we frame the problem analytically: we cast harness construction as a regret-minimization problem against an oracle reference, and decompose the regret into two complementary loss terms that map directly to actionable design axes.

Utility and regret. We define the utility of a harness C on a task x_t as

$$V(C, x_t) = \mathbb{E}_{a \sim \pi(\cdot \mid x_t, C)}[r(a, y(x_t))], \quad (1)$$

which is the expected reward of the harness-conditioned solver. The full-history utility is the corresponding ceiling under the capacity budget,

$V(\mathcal{H}_t, x_t) := \sup_{C: |C| \leq K} V(C, x_t)$, that is, the best utility attainable by any bounded harness on x_t . The regret of the evolver-constructed harness is then

$$\text{Regret}(\varphi, x_t) = V(\mathcal{H}_t, x_t) - V(\varphi(\mathcal{H}_t), x_t) \geq 0, \quad (2)$$

non-negative since $\varphi(\mathcal{H}_t)$ is one element of the supremum’s domain. Operationally, this ceiling matches what a solver granted $\mathcal{H}_t$ directly, under the same compute and tool budget as φ , could attain by reconstructing any candidate harness on the fly.

Solve-time optimal harness. Fix the evolver class Φ and define the solve-time optimal harness for a particular task:

$$C_\Phi^*(x_t) = \arg \max_{\substack{C = \varphi(\mathcal{H}_t, x_t), \varphi \in \Phi \\ |C| \leq K}} V(C, x_t). \quad (3)$$

This is an hypothetical oracle reference: it is the best harness the evolver class Φ could produce if it were allowed to condition on the incoming task x_t at solve time. A deployed evolver φ commits to one harness $\varphi(\mathcal{H}_t)$ before x_t is observed; using C_Φ^* as pivot between $V(\mathcal{H}_t, x_t)$ and $V(\varphi(\mathcal{H}_t), x_t)$ yields the following decomposition.

Proposition 1 (Regret Decomposition). *For any deployed evolver $\varphi \in \Phi$,*

$$\mathbb{E}_{x_t}[\text{Regret}(\varphi, x_t)] = L_{\text{evo}}(\Phi) + L_{\text{adapt}}(\varphi), \quad (4)$$

where

$$L_{\text{evo}}(\Phi) = \mathbb{E}_{x_t}[V(\mathcal{H}_t, x_t) - V(C_\Phi^*(x_t), x_t)], \quad (5)$$

$$L_{\text{adapt}}(\varphi) = \mathbb{E}_{x_t}[V(C_\Phi^*(x_t), x_t) - V(\varphi(\mathcal{H}_t), x_t)]. \quad (6)$$

L_{evo} is the *evolution loss*: it reflects the evolver class Φ ’s capability gap, regardless of how many cycles the evolver runs. A single-agent prompt editor cannot produce multi-file infrastructure; that ceiling is structural, not a matter of effort. Reducing L_{evo} therefore requires pursuing more capable evolver systems with access to broader control and feedbacks. L_{adapt} is the *adaptation loss*: the oracle builds the optimal harness per task, but a deployed φ commits to one harness before seeing x_t . Therefore, even with an optimal evolver system, L_{adapt} exists as long as task heterogeneity persists.

Human-in-the-loop as a third axis. The decomposition assumes $\mathcal{H}_t$ contains relevant signal for

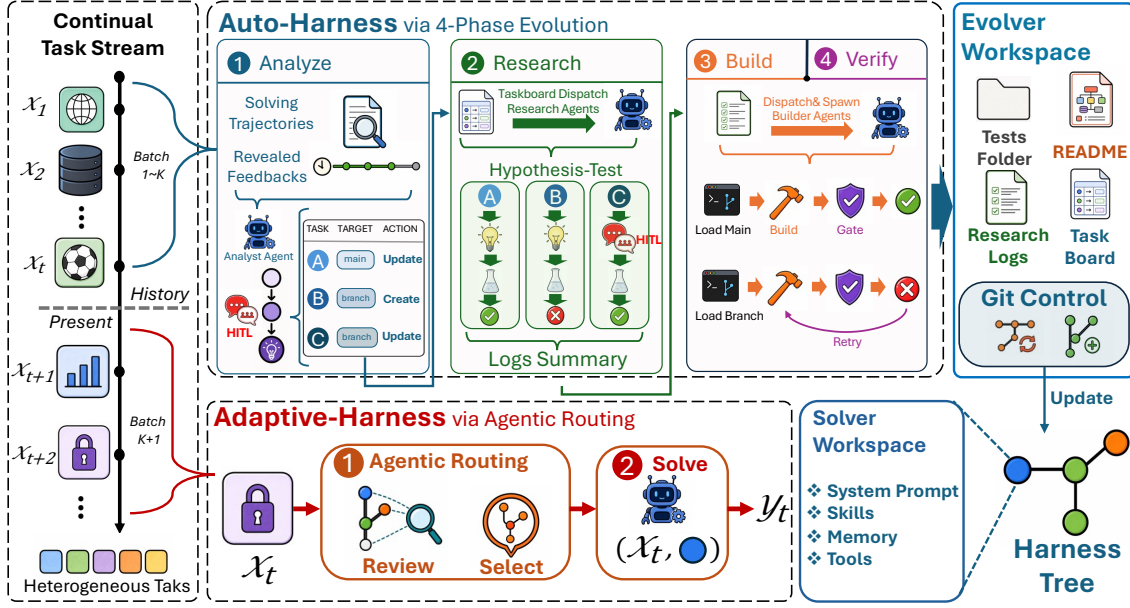


Figure 3: Overview of the Adaptive Auto-Harness system. **Bottom:** the multi-agent evolver constructs and refines a harness tree across cycles via four phases (Analyze → Research → Build → Verify) with a persistent cross-cycle workspace and temporal-reveal feedback. **Top:** at solve time, a router agent reads each branch’s workspace via git show and routes the incoming task x_t to the most suitable branch. Two human-in-the-loop hooks (task-board steering and research-phase assistance) trigger only when the evolver’s history lacks relevant signal.

x_t . When it does not, the regret decomposition no longer applies; we address this case via a third axis outside $L_{\text{evo}} + L_{\text{adapt}}$, the human-in-the-loop channel (§3.5).

3.3 Sustained Auto-Harness via Multi-Agent Evolution

The analytical decomposition identifies L_{evo} as the loss from harness capabilities that the evolver class cannot construct from history. We reduce this loss by expanding Φ with a stateful four-phase multi-agent evolver, temporal-reveal feedback, and cross-cycle memory (Figure 3). This design targets the unbounded-stream failure where a single-agent evolver must absorb growing trajectories, delayed labels, and prior research within one context window. Concretely, we address three structural limitations:

(1) **Multi-agent with distinct roles and objectives.** Because unbounded streams (D1) keep expanding the trajectory history an evolver must interpret, existing single-agent evolvers must fit analysis, research, implementation, and verification into one context window. We decompose evolution into four phases (Analyze → Research → Build → Verify), each with a dedicated objective and full context budget. This eliminates the single-window bottleneck and lets parallel Researchers explore disjoint

hypotheses without premature convergence.

(2) **Temporal-reveal feedback.** Under unbounded streams (D1), labels arrive asynchronously (e.g., a prediction market resolves days after the trade). We implement a *temporal-reveal gate* that surfaces each task’s ground-truth label to the evolver only after its resolution date, providing a proper streaming feedback signal without leaking future information.

(3) **Persistent cross-cycle state.** We provide the evolver with a dedicated workspace that persists across cycles, containing a task board (prioritised failure analysis), research logs (tested hypotheses with pass/fail verdicts), architecture documentation, and verification tests. This cross-cycle memory enables the evolver to refine its construction ability over time and build upon prior evolution experience rather than restarting from scratch.

3.4 Solve-Time Adaptation via Harness-Tree Routing

The analytical decomposition identifies L_{adapt} as the loss from committing to one dense harness before observing the incoming task’s context. We reduce this loss by shifting heavy adaptation into evolution time: the evolver constructs a *harness tree* of specialized harnesses, and solve time only requires a lightweight routing decision (Figure 3, top panel). This design targets task heterogeneity

and non-stationarity, where a single dense harness can underfit specialized task contexts even when the evolver is capable.

We instantiate this with two designs (Figure 3, top panel). **(1) Branching harness tree (built at evolution time).** The solver’s workspace is a git repository. The evolver constructs regime-specific branches (e.g., branch/crypto-classical, branch/binary-reversing) during evolution, each carrying its own prompt, skills, and tool registry; git provides versioning, isolation, and lineage tracking across branches. **(2) Agentic routing (executed at solve time).** A router agent reads each branch’s workspace via `git show` and selects the branch given x_t ’s context; the solver then checks out that branch and executes.

3.5 Human-in-the-Loop Channel

A third failure lies outside $L_{\text{evo}} + L_{\text{adapt}}$: some tasks require harness absent from $\mathcal{H}_t$ signal, such as API credentials, novel web sources, or proprietary endpoints. In this experience-insufficient setting, neither a stronger evolver nor solve-time routing can recover the missing signal. We address it with a human-in-the-loop channel that augments $\mathcal{H}_t$ through structurally triggered steering hooks (Figure 3). This design targets open-ended streams where new access requirements appear before autonomous evolution has relevant evidence. **(1) Task-board steering.** After the Analyst updates the task board, a human may review it to add entries, adjust priorities, or supply domain guidance and source access. This proactively steers the subsequent research cycle with direction the evolver cannot derive from trajectories alone. **(2) Interactive assistance during research.** When a Researcher hits a barrier mid-execution that requires human intervention (e.g., an authentication wall), the hook prompts the human in real time. This reactively unblocks the research agent at the point of failure.

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate on three open-ended task streams (Table 1): *PolyBench* for prediction markets (Cheng et al., 2026), *CTF-Dojo* for security challenges (Zhuo et al., 2025), and *FutureX* for event forecasting (Zeng et al., 2025). All three enforce strict temporal order and covers the three dimensions of challenge. Details and non-stationarity

Table 1: Benchmark statistics for the three chronological task streams.

Bench.	Tasks	Span	Resolution	Domain
PolyBench	5,075	Feb 6–22, 2026	Day	Prediction markets
CTF-Dojo	261	2011–2024	Year	Security
FutureX	503	Jan–Apr 2026	Week	Forecasting

diagnostics are in Appendix A and C.

Baselines. We compare with no-evolution runs with different solver agents using **Sonnet-4.6** (Anthropic, 2025b), **DeepSeek-V3.2** (DeepSeek-AI, 2025), **Claude Haiku** (Anthropic, 2025a), **textbfGLM-4.7** (Z.ai, 2025), and **Kimi-K2.5** (Moonshot AI, 2026). We compare with five auto-harness baselines: **A-Evolve** (Lin et al., 2026), **GEPA** (Agrawal et al., 2025), **Meta-Harness** (Lee et al., 2026), **Continual Harness**, and **SkilIOS**. We also compare with one human-designed system **OctoTools** (Lu et al., 2025).

Solver and evolver. We use Claude Sonnet 4.6 as the solver for all experiments unless specified, and use Claude Opus 4.6 as the evolver, both at temperature 0 to attribute gains to the evolution algorithm rather than sampling noise; the no-evolution controls additionally report base-agent results for Haiku 4.5, DeepSeek-V3.2, Kimi-K2.5, and GLM-4.7. All algorithms share the same batch size (100/20/20 for PolyBench/CTF-Dojo/FutureX), batch loop, and temporal-reveal gate; only the evolution algorithm varies.

Metrics. For PolyBench, we report three complementary metrics: *Coverage*, the fraction of all 5,075 markets the agent traded; *Accuracy*, the fraction of all markets traded correctly; and *Return*, defined as $\text{Coverage} \times \text{CWR}$, where CWR is the confidence-weighted portfolio profit-to-investment ratio over traded markets. For CTF-Dojo and FutureX, we report the official pass rate (Pass@1) defined by the original benchmarks (Zhuo et al., 2025; Zeng et al., 2025). We also report *lift* in the figures as the difference with baselines. Full definitions are in Appendix A.

Research questions. **RQ1** (§4.2): How does Adaptive Auto-Harness compare against existing auto-harness systems on open-ended task streams? **RQ2** (§4.3): How does Adaptive Auto-Harness address benchmark-specific bottlenecks? **RQ3** (§4.4): Does stateful multi-agent evolution with evaluation feedback provide additional gains? **RQ4** (§4.5): Can solve-time routing effectively leverage specialized harness branches? **RQ5** (§4.6): Can human

Table 2: Main comparison across no-evolution agents, auto-harness baselines, a human-designed system, and our three variants. PolyBench reports Coverage / Accuracy / Return (coverage-scaled CWR, in %); CTF-Dojo and FutureX report the official Pass@1. Bold marks the best result per row and underline marks second best.

		No evolution					Auto-harness baselines					Human	Ours		
Benchmark	Metric	Sonnet	Haiku	DeepSeek	Kimi	GLM	A-Evolve	GEPA	Meta Harness	Cont. Harness	SkillOS	OctoTools	Multi agent	Adaptive	Full System
PolyBench	Accuracy $\uparrow$	22.2	15.2	1.4	14.0	14.0	18.4	13.4	50.8	8.5	21.4	40.0	<u>79.8</u>	77.4	80.9
	Return $\uparrow$	+1.7	+88.0	+16.4	-2.0	+10.1	+7.2	+0.2	+320	+1.7	+3.6	+20.4	<u>+351</u>	+352	+330
CTF-Dojo	Pass $\uparrow$	37.2	23.8	26.1	24.5	12.6	45.2	42.9	41.0	25.7	29.5	38.3	<u>47.9</u>	46.0	50.2
FutureX	Pass $\uparrow$	31.0	31.0	31.2	27.8	30.8	<u>47.5</u>	28.2	29.4	31.8	29.8	25.6	49.5	44.1	47.3

steering help under insufficient history signal?

4.2 Comparison with Baselines

Prior systems specialize on one metric cluster.

A-Evolve leads the two pass-rate streams (45.2% CTF-Dojo, 47.5% FutureX) but covers only 21.1% of PolyBench markets. Meta-Harness leads all three PolyBench metrics (55.3% Coverage, 50.8% Accuracy, +320% Return) but falls below the no-evolution Sonnet baseline on FutureX (29.4% vs. 31.0%). Base solvers stay below 32.6% PolyBench Coverage on every model, and OctoTools, the frozen human-designed system, places third on PolyBench Return but does not lead any row. The pass-rate cluster and the portfolio cluster therefore sit in different baselines.

Our three variants jointly lead all five metrics.

Without HITL intervention, the Full System combines multi-agent evolution with solve-time routing and reaches 97.9% PolyBench Coverage, 80.9% Accuracy, and 50.2% CTF-Dojo Pass. The Multi-agent variant leads FutureX at 49.5%, where evolving the right source and tooling matters more than per-task routing. The Adaptive variant leads PolyBench Return at +352%, where matching each market to a specialized strategy matters more than wider coverage.

4.3 Benchmark Bottlenecks

RQ2 asks whether Adaptive Auto-Harness targets the bottlenecks that limit each benchmark. We organize the analysis around the two losses introduced in §3.2: the *evolution loss* L_{evo} , the gap from capabilities the evolver class cannot construct; and the *adaptation loss* L_{adapt} , the gap from committing to a single harness across heterogeneous tasks.

Evolution bottlenecks (L_{evo}) differ across benchmarks. Figure 4 plots a benchmark-specific stress axis chosen to expose the most predictive capability. The PolyBench panel plots mean stated confidence against market consensus, defined as the implied probability of the favored outcome from Polymar-

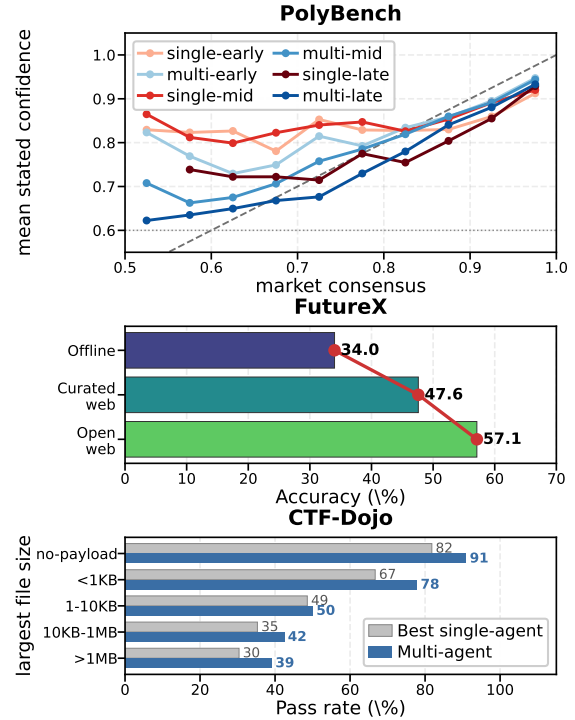


Figure 4: L_{evo} evidence across benchmark-specific bottlenecks. PolyBench stresses confidence calibration, FutureX stresses web-retrieval access, and CTF-Dojo stresses payload handling of different file sizes.

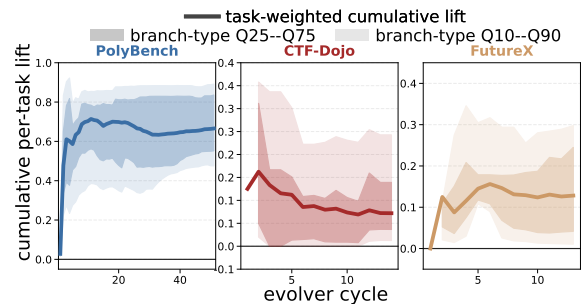


Figure 5: L_{adapt} evidence from branch specialization. Curves show cumulative routing lift over the baselines, while shaded bands show harness branch performance spread across cycles.

ket prices; a well-calibrated harness tracks the diagonal. The multi-agent variant does so, while single-agent variants stay flat and over-state confidence on

low-consensus markets, suggesting that the binding capability is consensus-aware confidence calibration rather than raw prediction skill. The FutureX panel plots pass rate against three retrieval tiers, ranging from offline to date-filtered Wikipedia plus DuckDuckGo to unrestricted DuckDuckGo. Pass rate increases monotonically from 34.0% to 47.6% to 57.1%, identifying source acquisition rather than reasoning as the binding capability. The CTF-Dojo panel plots pass rate against the largest challenge file size, binned into five tiers from no-payload to >1MB. The best single-agent variant declines from 81.8% to 30.4%, and the multi-agent variant declines from 90.9% to 39.1% while retaining roughly a 9-point margin throughout, indicating that payload-handling infrastructure becomes the binding capability as inputs grow and that the multi-agent evolver mitigates but does not eliminate this bottleneck. The binding capability therefore differs by benchmark, so no single fixed harness suffices. **Adaptation bottlenecks (L_{adapt}) remain after evolution.** Figure 5 reports per-task routing lift over evolver cycles, where lift on each task is the routed-harness score minus the median score across single-committed-harness baselines. The solid curve is the within-cycle mean lift and the shaded band is performance variation across different branches; if a single committed harness were sufficient, the mean lift would approach zero as evolution progresses. We observe instead that the mean lift and variation remains positive across all cycles on all three benchmarks.

Takeaway. Although streams expose different bottlenecks, they can be addressed under the same principle: reduce evolution loss for missing harness capabilities and adaptation loss for task-specific fit.

4.4 Stateful Multi-Agent Evolution

RQ3 asks whether the four-role evolver in §3.3 (Analyst → parallel Researchers → Builder → Verifier) improves over a single-agent evolver, and whether its state channels matter. Figure 6 compares the full system with variants that remove temporal-reveal feedback or cross-cycle memory. The full system improves over the single-agent evolver on all three benchmarks: 20.3 → 44.3 CWR on PolyBench, 38% → 43% on CTF-Dojo, and 38% → 44% on FutureX. Removing memory causes the broadest degradation (2.6 CWR on PolyBench, 30% on CTF-Dojo, and 38% on FutureX), while removing feedback mainly hurts PolyBench (7.5 CWR), where outcomes resolve after trading. Overall, the

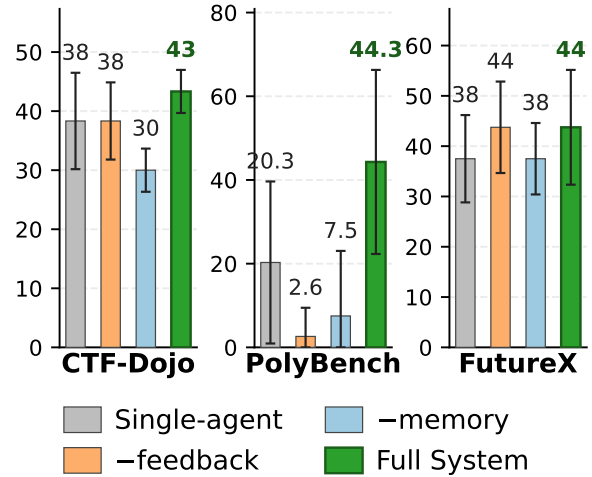


Figure 6: Ablations for multi-agent evolution. Removing evaluation feedback or cross-cycle memory degrades performance relative to the full stateful system.

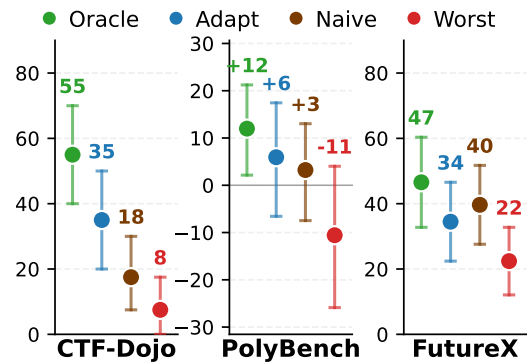


Figure 7: Solve-time routing evaluation on a evolved harness tree. Oracle reports the best branch performance per task, Adapt uses our agentic router, Naive always uses main workspace, and Worst reports the worst branch performance per task.

four-role evolver is strongest when paired with persistent state: memory preserves cross-cycle search, and feedback turns resolved outcomes into later evolution signal.

4.5 Solve-Time Routing on the Harness Tree

RQ4 asks whether solve-time routing can leverage the specialized harness tree. We replay all branches on the same tasks and compare *Oracle* (best branch per task), *Adapt* (our router), *Naive* (main), and *Worst* (worst branch per task). Figure 7 shows clear Oracle-to-Naive headroom: 37 points on CTF-Dojo, +9 CWR on PolyBench, and 7 points on FutureX. Adapt captures the gap on CTF-Dojo and PolyBench, but trails Naive on FutureX, where source acquisition dominates routing choice.

Takeaway. Harness specialization creates headroom across streams, but FutureX shows that rout-

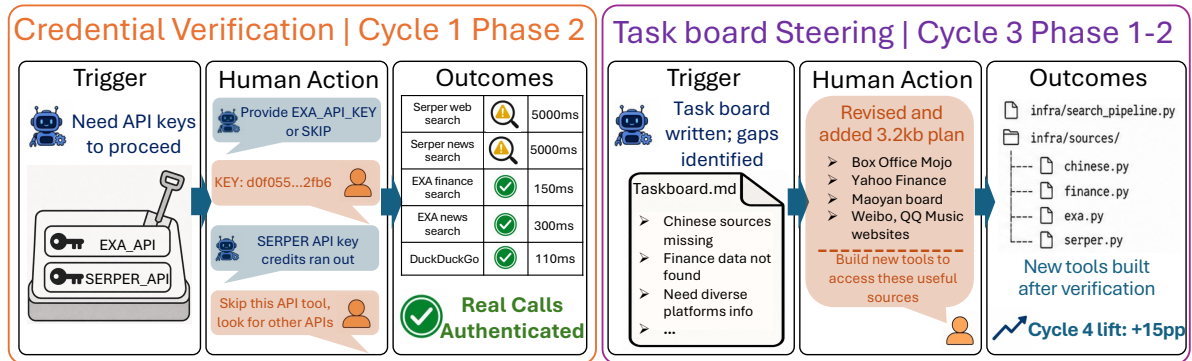
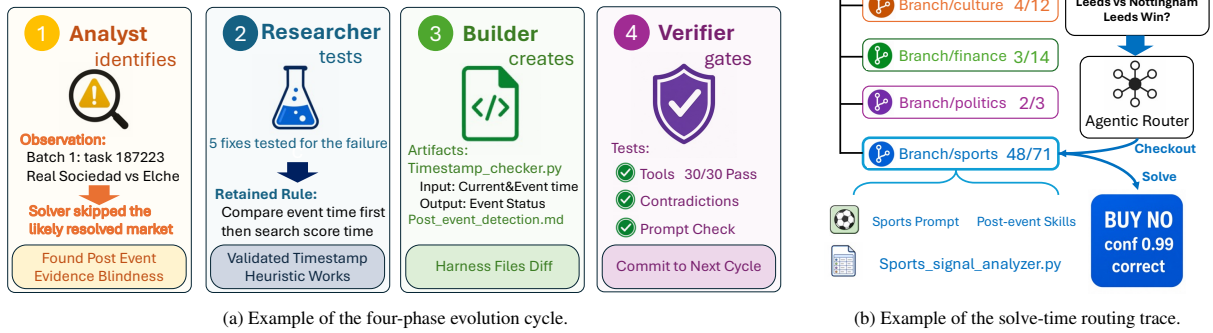


Figure 8: Extracted trajectories of the designed multi-agent evolution, agentic routing, and Human-in-the-Loop.

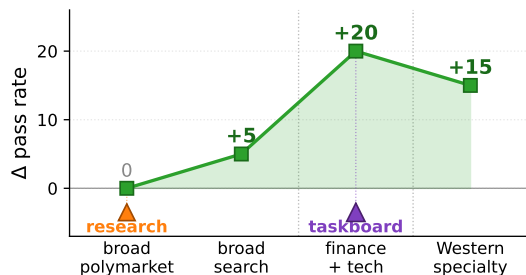


Figure 9: FutureX pass-rate lift over four task slices under two human-steering hooks. The orange triangle marks research-phase steering and the purple triangle marks task-board steering. Lift is measured against the no-HITL run.

ing cannot compensate when source acquisition dominates branch choice.

4.6 Human Steering for Auto-Harnessing

RQ5 asks whether human steering helps when history lacks the source or access signal needed for evolution. Since all previous experiment does not apply HITL to ensure fairness, we evaluate this on FutureX and restrict human input to two hooks: *research-phase steering* supplies credentials when research is blocked, and *task-board steering* adds source directions the evolver cannot infer (Figure 8c). Figure 9 shows the slice-level effect: lift

is 0 on broad polymarket questions, rises to +5 on broad search-dependent questions, peaks at +20 on the directly targeted finance&tech slice, and remains +15 on adjacent Western-specialty questions. The pattern indicates that HITL helps when it injects the missing external signal, not generic human advice.

Takeaway. Human steering is most useful when the missing ingredient is external source knowledge rather than additional autonomous evolution.

5 Conclusions

Open-ended task streams expose three challenges for auto-harness deployment: unbounded task arrival, heterogeneous tasks, and non-stationarity. Adaptive Auto-Harness addresses them by pairing sustained harness construction with solve-time task adaptation. The decomposition into evolution loss and adaptation loss explains why a single repeatedly updated harness is insufficient: the system must construct missing capabilities from stream evidence while selecting a suitable specialized branch for each task. Across three streams, the experiments show complementarity: evolution builds capabilities, routing exploits specialization, and human steering supplies external signals absent from history.

6 Limitations

Benchmark coverage. We evaluate finite prefixes of three open-ended task streams: prediction markets, cybersecurity challenges, and event forecasting. These domains cover unbounded streams, task heterogeneity, and distributional non-stationarity, but the same framing should be tested further on additional deployment streams such as robotics, biomedical reasoning, and long-form writing.

Diagnostic losses. The evolution loss L_{evo} and adaptation loss L_{adapt} are analytical quantities, not directly estimated oracle losses. Our experiments diagnose them through bottleneck analyses, ablations, and routing controls rather than through a formal estimator of the oracle harness.

7 Ethical Considerations

We use public or research benchmark tasks and do not introduce private user data. CTF-Dojo runs only inside isolated benchmark containers and does not target real systems. Human steering is limited to source guidance, task-board edits, and credential decisions; humans do not label answers or choose solver branches. Released artifacts should preserve sandbox restrictions and exclude credentials or provider secrets.

8 AI Usage Statement

We used AI assistants to refine the writing of this paper and to accelerate debugging and analysis during implementation and evaluation. The authors reviewed and are responsible for all content and reported results.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, and 1 others. 2025. Gega: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*.

Anthropic. 2025a. Claude haiku 4.5. <https://www.anthropic.com/claude/haiku>.

Anthropic. 2025b. Claude sonnet 4.6. <https://www.anthropic.com/claude/sonnet>.

Shai Ben-David, John Blitzer, Koby Crammer, Alex Kulesza, Fernando Pereira, and Jennifer Wortman Vaughan. 2010. A theory of learning from different domains. *Machine Learning*, 79:151–175.

Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. 2020. Dark experience for general continual learning: A strong, simple baseline. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 15920–15930.

Pu Cheng, Juncheng Liu, and Yunshen Long. 2026. Polybench: Benchmarking llm forecasting and trading capabilities on live prediction market data. *arXiv preprint arXiv:2604.14199*.

DeepSeek-AI. 2025. Deepseek-v3.2 technical report. *arXiv preprint arXiv:2512.02556*.

Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. 2016. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59):1–35.

Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.

Seth Karten, Joel Zhang, Tersoo Upaa Jr, Ruirong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vondrahal. 2026. Continual harness: Online adaptation for self-improving foundation agents. *arXiv preprint arXiv:2605.09998*.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*.

Jian Liang, Dapeng Hu, and Jiashi Feng. 2020. Do we really need to access the source data? source hypothesis transfer for unsupervised domain adaptation. In *International Conference on Machine Learning (ICML)*.

Minhua Lin, Hanqing Lu, Zhan Shi, Bing He, Rui Mao, Zhiwei Zhang, Zongyu Wu, Xianfeng Tang, Hui Liu, Zhenwei Dai, and 1 others. 2026. Position: Agentic evolution is the path to evolving llms. *arXiv preprint arXiv:2602.00359*.

Pan Lu, Bowen Chen, Sheng Liu, Rahul Thapa, Joseph Boen, and James Zou. 2025. Octotools: An agentic framework with extensible tools for complex reasoning. *arXiv preprint arXiv:2502.11271*.

Rui Miao, Babak Shahbaba, and Annie Qu. 2025. Reinforcement learning for individual optimal policy from heterogeneous data. *Annals of statistics*, 53(4):1513.

Moonshot AI. 2026. Kimi k2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*.

Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, and 1 others. 2026. Skillos: Learning skill curation for self-evolving agents. *arXiv preprint arXiv:2605.06614*.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*.

Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. 2021. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations (ICLR)*.

Zifeng Wang, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer G. Dy, and Tomas Pfister. 2022a. Dualprompt: Complementary prompting for rehearsal-free continual learning. In *European Conference on Computer Vision (ECCV)*, pages 631–648.

Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer G. Dy, and Tomas Pfister. 2022b. Learning to prompt for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 139–149.

Z.ai. 2025. Glm-4.7. <https://www.z.ai/>.

Zhiyuan Zeng, Jiashuo Liu, Siyuan Chen, Tianci He, Yali Liao, Yixiao Tian, Jinpeng Wang, Zaiyuan Wang, Yang Yang, Lingyue Yin, and 1 others. 2025. Futurex: An advanced live benchmark for llm agents in future prediction. *arXiv preprint arXiv:2508.11987*.

Terry Yue Zhuo, Dingmin Wang, Hantian Ding, Varun Kumar, and Zijian Wang. 2025. Training language model agents to find vulnerabilities with ctf-doj. *arXiv preprint arXiv:2508.18370*.

A Benchmark and Evaluation Details

Across all three benchmarks, tasks are evaluated in chronological order. The solver receives task x_i with only the information available before its release time, while the evolver receives outcome labels only after the corresponding resolution time. The table generator loads the first record for each instance_id, so retries or duplicate logs do not change the reported metrics. The three datasets are finite evaluated prefixes of unbounded task streams and jointly cover the three deployment challenges in §1: stream continuation (D1), within-stream heterogeneity (D2), and temporal shift (D3).

A.1 PolyBench

Source and composition. PolyBench is a Polymarket-derived prediction-market stream with 5,075 binary-outcome tasks from Feb 6–22, 2026. The stream spans politics, sports, finance, crypto, and entertainment markets, and each task is resolved against the official market outcome.

Temporal ordering. For each market, we store the task release timestamp and the outcome resolution timestamp. Retrieval evidence is filtered by release time during solving. Resolved labels are hidden from evolution until the corresponding market has resolved.

Metrics. Let $\mathcal{T}$ be the set of executed trades, excluding gated tasks, empty decisions, and SKIP decisions. Let s_i indicate whether trade i is correct, b_i be its confidence-weighted investment, and g_i be its realized profit. The metrics reported for PolyBench are:

$$\text{Acc} = 100 \cdot \frac{1}{N} \sum_{i \in \mathcal{T}} s_i$$

$$\text{CWR} = 100 \cdot \frac{\sum_{i \in \mathcal{T}} g_i}{\sum_{i \in \mathcal{T}} b_i}$$

$$\text{Return} = \frac{|\mathcal{T}|}{N} \cdot \text{CWR}.$$

Accuracy therefore rewards both broad coverage and correct decisions, while Return scales the portfolio profit-to-investment ratio by the fraction of tasks actually traded.

A.2 CTF-Dojo

Source and composition. CTF-Dojo is a 261-challenge security-competition stream drawn from pwncollege/ctf-archive, chronologically ordered from 2011 to 2024. Challenges cover binary exploitation, web security, cryptography, reverse engineering, and forensics, exposing changes in challenge style and tooling over time.

Sandbox and verification. Each challenge runs inside a per-task Docker sandbox with constrained network policy. Flags are submitted as text and verified by SHA-256 hash comparison against the official flag.

Metrics. We report Pass@1, the percentage of challenges solved within the benchmark budget. All CTF-Dojo aggregate results use the same chronological ordering as the released stream.

A.3 FutureX

Source and composition. FutureX is a 503-question event-forecasting stream over 82 days from Jan–Apr 2026, drawn from FutureX-Past.

Questions cover finance, technology, geopolitics, and entertainment, with both English and Chinese-language variants. The zh-finance slice requires source discovery beyond default English-only retrieval.

Temporal retrieval and filtering. FutureX tasks are historical, but web pages and search indices continue to change after the event. To avoid label leakage, each task is solved with a per-task cutoff date derived from its temporal metadata. In strict built-in retrieval, Wikipedia content is fetched through the revision API using the latest revision before the cutoff, DuckDuckGo results are fetched and filtered by extracted publication dates from htmldate and URL patterns, and structured economic series are queried with observation and real-time endpoints capped at the cutoff. For evolved tools executed through the sandbox, live command output is passed through an LLM temporal filter before the solver sees it: the filter receives the task, cutoff date, and retrieved content, then either returns CLEAN or replaces post-cutoff values, rows, and snippets with [REDACTED]. This preserves pre-cutoff evidence while blocking dynamic web content that would reveal the resolved answer.

Metrics. We report Pass@1 under the official FutureX criterion. Slice-level human-steering analyses use the same pass criterion and aggregate tasks by the groups shown in Figure 9.

B Implementation and Reproducibility Details

Execution protocol. All main runs use provider-hosted LLM APIs with native tool calling and the same chronological task order used by the benchmarks. Reported numbers are generated from the corresponding results.jsonl files. Closed provider-hosted models do not expose parameter counts; we therefore report the evaluated task counts and evolution cycles rather than GPU-hours. The full-system runs contain 5,075/261/503 solve trajectories and 51/14/26 evolution cycles for PolyBench/CTF-Dojo/FutureX, respectively.

Compute and resources. All model inference is performed through provider-hosted APIs; the primary Adaptive Auto-Harness runs use Claude Sonnet for solving and Claude Opus for evolution, with other provider-hosted models used only for the corresponding baseline rows. We do not train or fine-tune model weights, and no local GPU compute is used for model optimization. Local compute

is used for orchestration, result aggregation, figure generation, and Docker-based benchmark execution, including the per-task CTF-Dojo and FutureX sandboxes. Because the evaluated closed models do not disclose parameter counts, task counts and evolution cycles are the main compute descriptors.

Temporal reveal. Each task stores a release timestamp and, when available, a resolution timestamp. Solver calls are filtered against the release time. Evolution cycles receive the trajectory immediately but receive outcome feedback only after the task has resolved, so unresolved tasks remain unlabeled history rather than leaked supervision.

Workspace artifacts. The evolver workspace persists a task board, research logs, verifier notes, tests, and architecture notes across cycles. These artifacts are separate from the solver workspace: the evolver may update harness files during evolution, while the solve-time router only inspects branch metadata and selects a branch for the incoming task.

Branch replay protocol. For routing analysis, every branch in the evolved harness tree is replayed on the same task set before constructing oracle and worst controls. Oracle and worst are therefore post-hoc diagnostic bounds; the deployed router only sees task context and branch metadata, not labels or branch outcomes.

Human-steering records. Human-steering events are author-provided system interventions rather than recruited human-subject annotations. Each event is logged with the triggering phase, requested external signal, and workspace location where the response is recorded. This makes steering auditable and keeps human input as source or access guidance rather than answer labels.

C Benchmark Non-Stationarity Diagnostics

Figures 10–12 provide descriptive diagnostics for the temporal structure of the three streams. These plots are not used as evaluation metrics; instead, they show why the benchmarks are not static IID pools and why a harness fitted to earlier observations can become mismatched to later tasks. Most panels are computed from task metadata, task text, or benchmark-side properties; panels that use outcomes or a baseline solver are included only as descriptive solvability proxies.

PolyBench. Prediction markets shift in both difficulty and tradability over the evaluated period (Figure 10). Early markets are more often liquid

and already decisive: the fraction of tradeable markets drops from 97% early to 31% late, and the fraction whose maximum price exceeds 0.95 drops from 44% to 29%. At the same time, near-even markets increase from 18% to 35%, meaning later tasks more often require evidence beyond simply following a strong market consensus. The market-price correctness proxy also changes over time, from 84% early to 77% late. Together, these shifts make a fixed prediction-market strategy brittle: calibration, abstention, and evidence gathering must adapt as the stream moves from liquid and decisive markets toward thinner and more ambiguous ones.

CTF-Dojo. CTF-Dojo exposes a different form of non-stationarity: the stream expands into competitions and challenge conventions not present in the early history (Figure 11). The cumulative number of source competitions keeps increasing across the chronological order, and by the late stream the fraction of tasks from competitions unseen in the first third reaches 100%. The number of competitions represented in a 50-task window also varies substantially, so neighboring tasks can require different assumptions about file layout, scoring conventions, and intended exploitation style. The cross-competition score coefficient of variation further indicates that competitions are not interchangeable pools; each event can calibrate difficulty and challenge design differently. This motivates persistent construction of reusable security skills, but also cautions against treating early CTF experience as uniformly transferable.

FutureX. FutureX shifts along source, language, and difficulty dimensions (Figure 12). Batch-level baseline accuracy ranges from 20% to 80%, showing that chronological batches differ substantially in solvability. Later batches contain more Chinese-titled questions and more questions tied to platforms that are difficult to search directly, while the share of harder Level 3–4 questions increases sharply in the same region. Chinese-language answer requirements also appear mainly in later batches. These changes explain why FutureX stresses both construction and adaptation: the harness must acquire better source-finding and temporal retrieval behavior, while solve-time routing must select branches suited to the task’s language, source, and difficulty profile.

D Further Experiments

We include two supplemental diagnostics that clarify where the main gains come from without duplicating the RQ analyses in §4.

Evolver capability and construction budget. Figure 13 varies the evolver model and construction budget on CTF-Dojo. Stronger evolvers achieve higher pass rates, while additional budget mainly helps weaker evolvers and saturates for the strongest model.

Cross-domain workspace dilution. Figure 14 compares PolyBench CWR when using workspaces evolved on different domains. The PolyBench-specific workspace performs best, while the all-evolved workspace loses 57 points of CWR, showing that mixing heterogeneous experience can dilute domain-relevant harness structure.

The Decision Environment Shifts Over Time

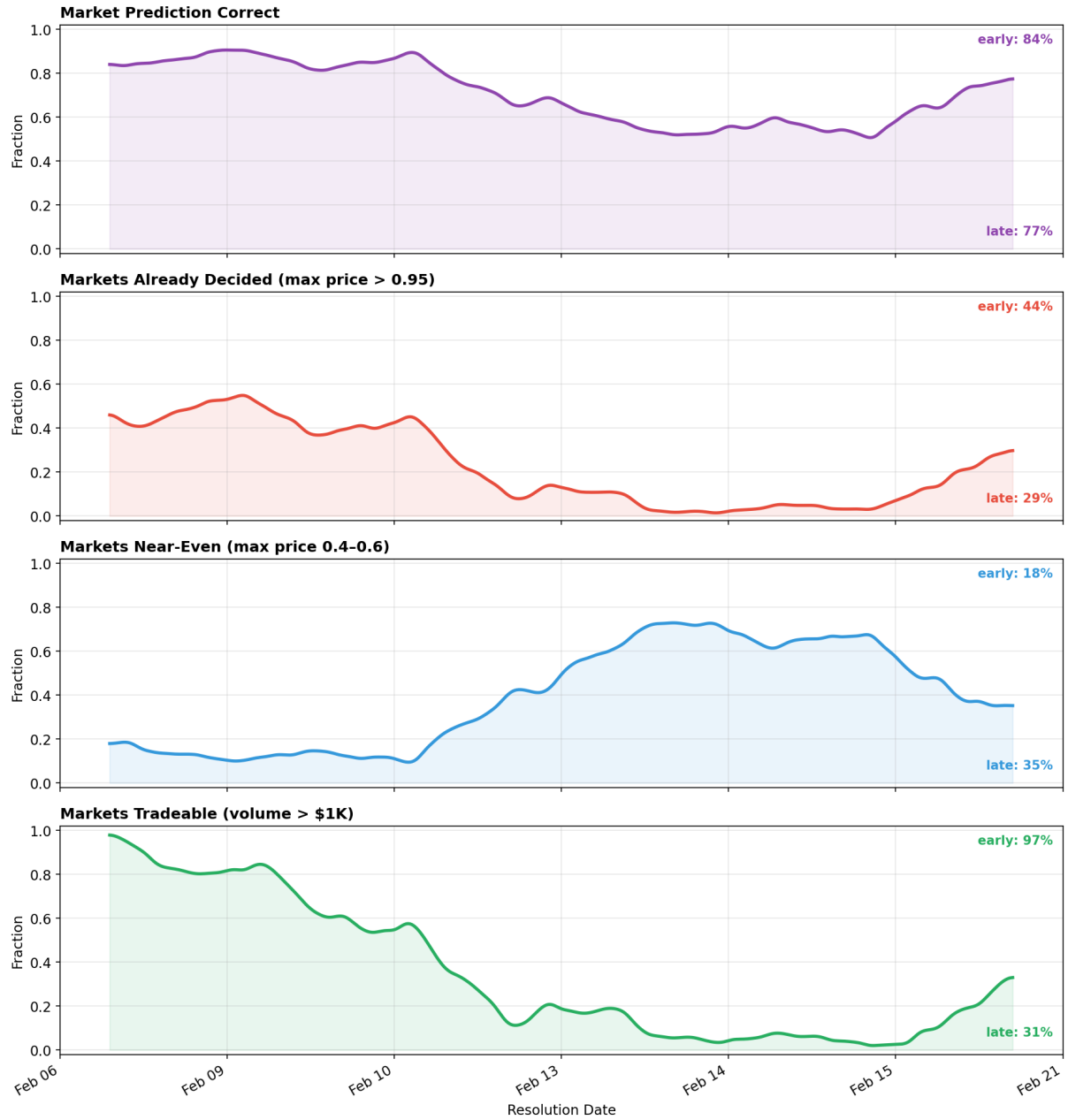


Figure 10: **PolyBench non-stationarity.** Market difficulty and tradability shift over time: later markets are less often decisive or liquid and more often near-even.

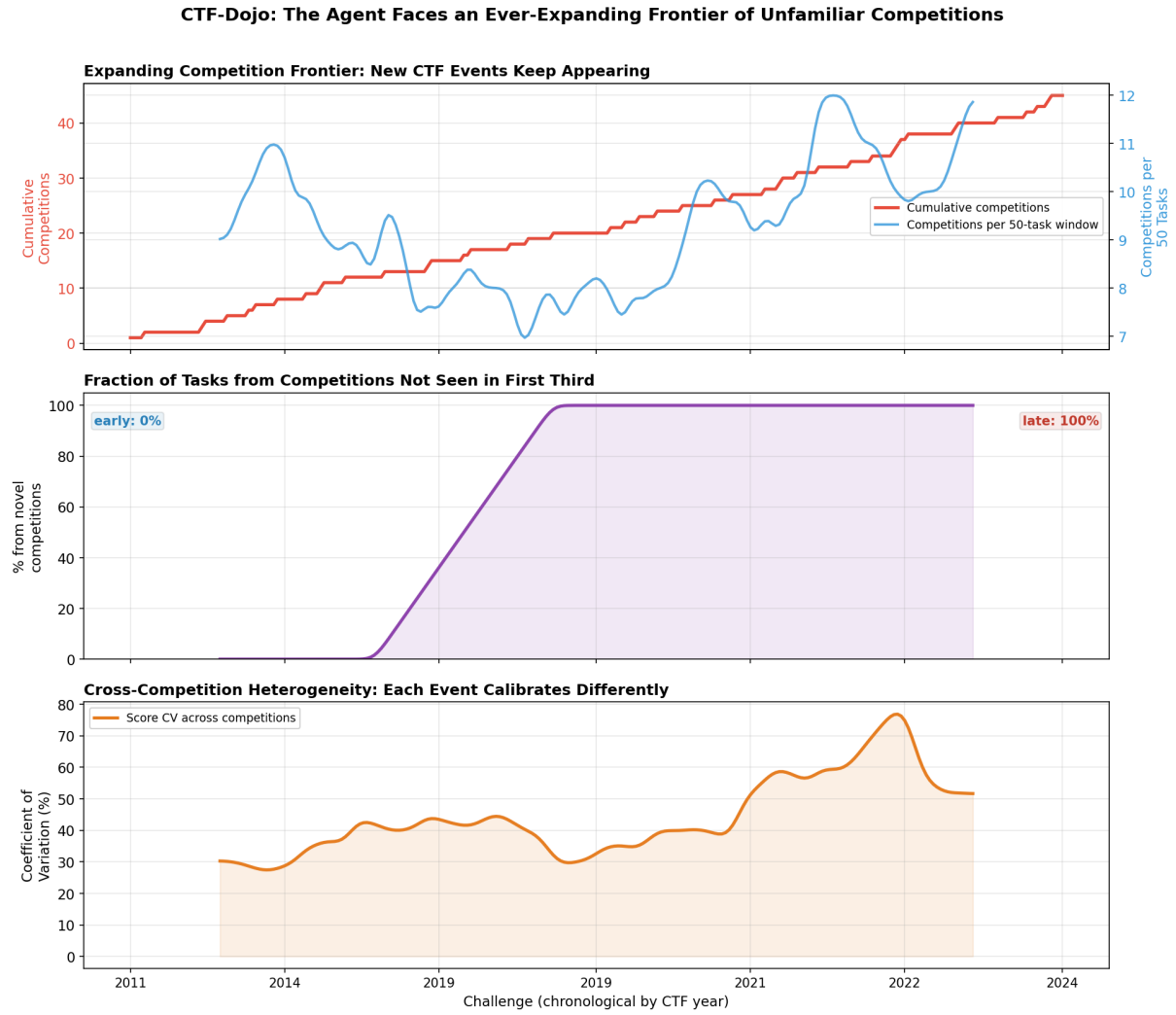


Figure 11: **CTF-Dojo non-stationarity.** The chronological stream keeps introducing new competitions and increases cross-competition variability, so early challenge experience is not uniformly transferable.

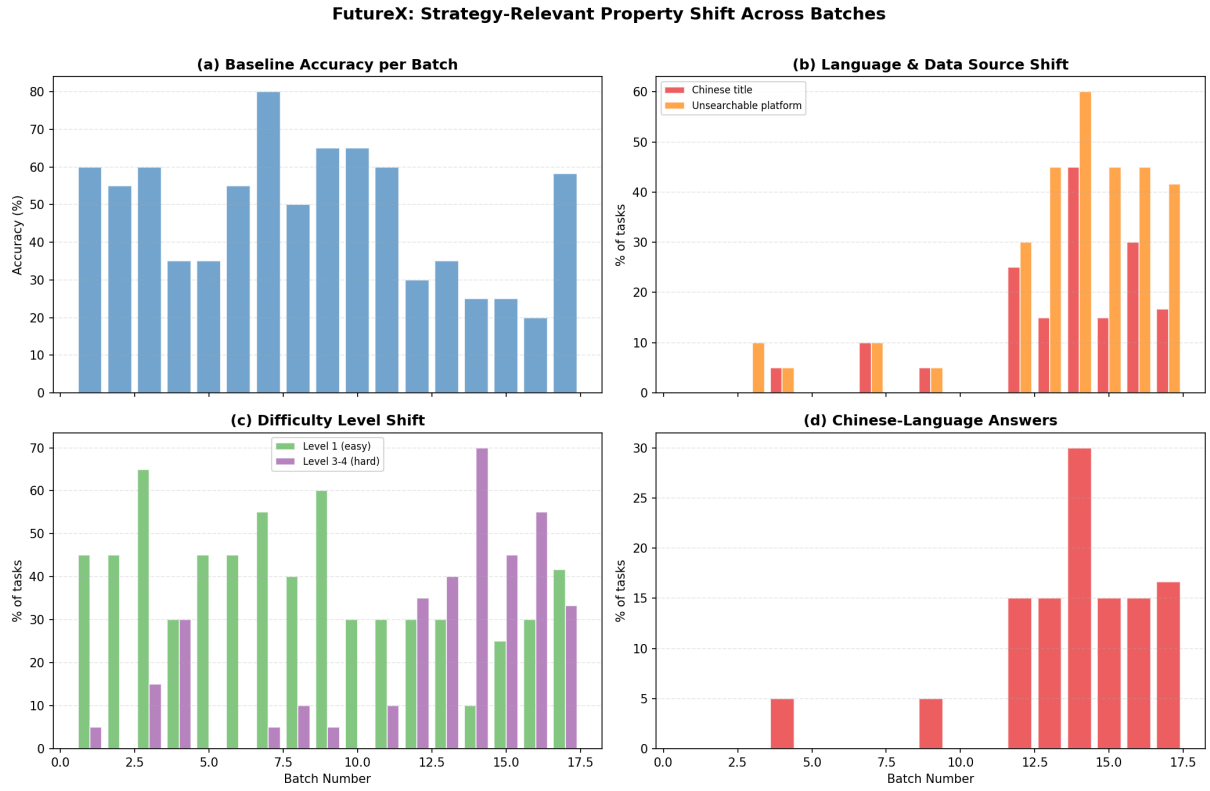


Figure 12: **FutureX non-stationarity.** Language, source accessibility, difficulty, and answer format shift across batches, creating solve-time harness mismatch when one static harness is reused for all tasks.

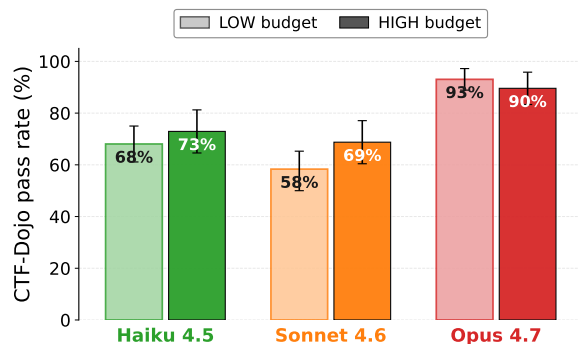


Figure 13: **Evolver capability on CTF-Dojo.** Pass rate improves with stronger evolver models; higher budget helps Haiku and Sonnet but gives little additional gain once Opus already reaches high performance.

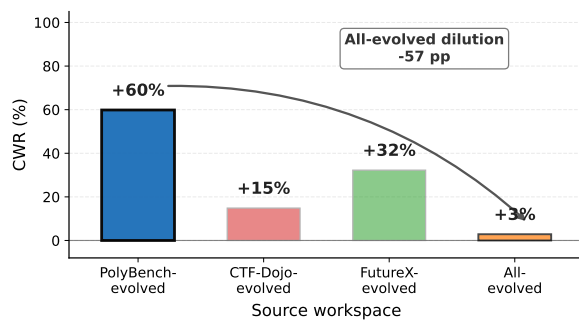


Figure 14: **PolyBench workspace dilution.** A PolyBench-evolved workspace reaches the highest CWR, while combining all evolved workspaces sharply reduces CWR, supporting the need for specialized harness branches rather than a single dense harness.

E Simplified Prompt Interfaces

The full prompts contain benchmark-specific paths and implementation details. We report simplified versions here to show the role structure and information flow used by Adaptive Auto-Harness.

Solve-time router

Input: the incoming task and branch summaries. **Instruction:** read each branch’s intended scope, match the task to positive and negative routing signals, and select exactly one harness branch. Use the main branch when multiple branches match weakly or no branch has clear evidence. Return a branch name, confidence, and short reason.

Analyst

Input: recent trajectories, revealed feedback, branch statistics, and the persistent evolution workspace. **Instruction:** identify recurring failure patterns, separate transferable fixes from branch-specific fixes, and write a task board with priorities and targets. Audit stale or over-specialized artifacts before proposing new changes.

Researcher

Input: an analyst-assigned failure pattern or benchmark bottleneck. **Instruction:** search for candidate tools, APIs, data sources, or implementation strategies; test whether each candidate works under the benchmark constraints; and record concise findings that the builder can directly use.

Builder

Input: the task board, verified research records, current solver workspace, and architecture notes. **Instruction:** implement only the changes supported by verified research, update prompts, tools, memory, or infrastructure as needed, and preserve existing working code unless a targeted replacement is justified.

Verifier

Input: the builder’s changes, target failure pattern, and benchmark constraints. **Instruction:** run focused tests before the changes go live, check that the implementation respects temporal filtering and sandbox rules, and return a pass/fail verdict with the concrete failure reason when retry is needed.

Human steering

Input: task-board gaps or research failures where history lacks the needed signal. **Instruction:** add source guidance, credential decisions, or priority edits without supplying answer labels. The steering is logged in the workspace so later evolution can inspect what external signal was provided.